ytLending清算文档

重要说明

本清算机器人采用**无状态轮询设计**，通过监听链上事件发现活跃借款人，并自动执行清算操作以保护协议免受坏账风险。

核心特性：

- **无状态设计**：无需数据库，所有数据从链上实时查询

- **轮询模式**：每 5 秒检查一次新区块

- **事件驱动**：监听 4 种事件发现活跃借款人

- **批量清算**：一次交易清算多个账户，节省 Gas

目录

1. [系统启动流程](#1-系统启动流程)
2. [主循环轮询流程](#2-主循环轮询流程)
3. [活跃地址获取流程](#3-活跃地址获取流程)
4. [清算检查流程](#4-清算检查流程)
5. [批量清算执行流程](#5-批量清算执行流程)
6. [重要参数说明](#6-重要参数说明)
7. [完整脚本](#7-完整脚本)
8. [运行和部署](#8-运行和部署)

---

1. 系统启动流程

启动命令：

代码块

```
npx hardhat run scripts/liquidation_bot/index.ts --network arbSepolia
```

启动步骤：

步骤 1 — 初始化 Hardhat 环境，读取网络配置

```
代码块
network: arbSepolia
chainId: 421614
rpcUrl:  https://arbitrum-sepolia.gateway.tenderly.co
```

步骤 2 — 读取部署信息（`deployments-lending.json`），加载合约地址

```
代码块
Lending Proxy:     0xCb4E7B1069F6C26A1c27523ce4c8dfD884552d1D
Base Token (USDC): 0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d

Collateral Assets:
  YT-A: 0x97204190B35D9895a7a47aa7BaC61ac08De3cF05
  YT-B: 0x181ef4011c35C4a2Fda08eBC5Cf509Ef58E553fF
  YT-C: 0xE9A5b9f3a2Eda4358f81d4E2eF4f3280A664e5B0

注: Price Feed 合约不再需要初始化
```

步骤 3 — 初始化签名者（从 `PRIVATE_KEY` 环境变量读取）

```
代码块
Liquidator Address: 0x...
ETH Balance: 0.5 ETH

要求:
  - 需要 ETH 支付 Gas 费用
  - 建议保持至少 0.1 ETH 余额
```

步骤 4 — 初始化合约实例

```
代码块
lendingContract = await ethers.getContractAt(
    'Lending',
    deployment.lendingProxy,
    signer
)
```

步骤 5 — 进入主循环

2. 主循环轮询流程

主循环参数：`LOOP_DELAY = 5000ms（5 秒）`

循环步骤：

1. 调用 `provider.getBlockNumber()` 获取当前区块号
2. 判断是否为新区块：
   - **新区块** → 执行清算逻辑（见下方步骤 3）
   - **已检查过** → 等待 5 秒后继续
3. 执行 `liquidateUnderwaterBorrowers(lendingContract, signer)`：
   - ① 获取活跃地址（查询事件）
   - ② 检查每个地址是否可清算
   - ③ 批量执行清算
4. 更新 `lastBlockNumber = currentBlockNumber`，循环回步骤 1

容错机制：

代码块

```
try {
    // 执行清算逻辑
} catch (error) {
    console.error('Error in main loop:', error)
    // 继续运行，不中断
}
// 无论成功或失败，都等待 5 秒后继续
await sleep(5000)
```

关键设计决策：

参数	值	说明
LOOP_DELAY	5000ms	轮询间隔
去重检查	lastBlockNumber	每个区块只处理一次，避免重复
容错机制	try-catch	异常不中断，继续运行

3. 活跃地址获取流程

函数：`getUniqueAddresses(lendingContract)`，`LOOKBACK_BLOCKS = 10000`

步骤 1 — 计算查询区块范围

```
代码块

currentBlock = 12345678
fromBlock    = 12345678 - 10000 = 12335678
toBlock      = 12345678

时间窗口（Arbitrum Sepolia，出块 ~1 秒）：
10000 区块 ≈ 2.7 小时
```

步骤 2 — 查询 4 种事件，提取活跃地址

事件	说明	提取字段
`Withdraw`	用户借款/提取 USDC	`event.args.src`、`event.args.dst`
`Supply`	用户存入 USDC	`event.args.from`、`event.args.dst`
`SupplyCollateral`	用户存入抵押品	`event.args.from`、`event.args.dst`
`WithdrawCollateral`	用户提取抵押品	`event.args.src`、`event.args.to`

步骤 3 — 合并去重

使用 `Set<string>` 自动去重，示例：

```
代码块

总事件数: 150 + 80 + 120 + 90 = 440 个
唯一地址数: 85 个（去重后）

覆盖率对比:
  Compound V3（仅 Withdraw）: ~95%
  本机器人（4 种事件）:       ~98%
```

步骤 4 — 返回地址列表，进入清算检查

4. 清算检查流程

遍历所有活跃地址，对每个地址执行：

1. 调用 `lendingContract.isLiquidatable(address)` （链上计算）
2. 结果判断：

 - **false** → 跳过（账户健康）

 - **true** → 加入清算列表 `liquidatableAccounts[]`

3. 继续检查下一个地址，全部检查完毕后进入批量清算

---

5. 批量清算执行流程

清算列表示例： `[0xAlice, 0xBob, 0xCharlie]`，共 3 个可清算账户

步骤 1 — 检查列表是否为空

- 空列表 → 无清算，等待下一轮
- 非空 → 继续执行

****步骤 2 — 调用 `absorbMultiple()`****

代码块

```
function absorbMultiple(
    address absorber,       // 清算者地址（机器人）
    address[] accounts      // [0xAlice, 0xBob, 0xCharlie]
) external override nonReentrant

// 修饰符检查:
// ✓ nonReentrant - 防重入保护
// ✓ 未暂停检查
```

步骤 3 — 累积利息 （`accrueInternal()`）

- 更新 `supplyIndex` （存款利息）
- 更新 `borrowIndex` （借款利息）
- 确保清算时使用最新的债务余额

****步骤 4 — 对每个账户执行 `_absorbInternal()`****

以 Alice 为例：

1. 再次验证可清算性：`if (!isLiquidatable(Alice)) revert NotLiquidatable()`（防止抢跑）
2. 获取用户债务：`principal = userBasic[Alice].principal`（负数 = 借款，如 -1000e6）
3. 遍历所有抵押资产 `[YT-A, YT-B, YT-C]`，获取抵押数量
4. 计算抵押品价值：`collateralValue = collateralAmount × assetPrice × liquidationFactor(0.95)`
5. 转移抵押品：`userCollateral[Alice][asset] = 0`，`userCollateral[absorber][asset] += collateralAmount`
6. 清零债务：`userBasic[Alice].principal = 0`，`totalBorrowBase -= 1000e6`

结果： Alice 债务清零，抵押品转移给清算者，清算者获得 5% 折扣

步骤 5 — 继续清算 Bob、Charlie（相同流程）

步骤 6 — 发送交易并等待确认

代码块

```
const tx = await lendingContract.absorbMultiple(
    await signer.getAddress(),  // 清算者地址
    liquidatableAccounts        // [Alice, Bob, Charlie]
)
const receipt = await tx.wait()
```

清算结果示例：

代码块

```
✅ Liquidation successful!
Transaction: 0xabc123...
Gas used: 450,000
Block: 12345679

清算了 3 个账户:
  0xAlice:   $1,000 债务
  0xBob:     $500 债务
  0xCharlie: $2,000 债务

清算者获得的抵押品:
  YT-A: 500 个（估值 $1,750）
  YT-B: 800 个（估值 $1,600）
  总价值: $3,350（覆盖 $3,500 债务，5% 折扣）
```

6. 重要参数说明

6.1 轮询参数

参数名	值	单位	说明	调优建议
LOOP_DELAY	5000	毫秒	轮询间隔	测试网 5000ms / 主网高活跃 2000ms / 激进模式 1000ms（注意 RPC 限流）
LOOKBACK_BLOCKS	10000	区块	事件查询范围	Arbitrum 测试网 50000（~14小时）/ Arbitrum 主网 10000-20000（~3-6小时）/ 以太坊主网 10000（~33小时）

时间窗口参考：

代码块

```
Arbitrum（出块 ~1 秒）：
  10000 区块 ≈ 2.7 小时
  50000 区块 ≈ 14 小时
 100000 区块 ≈ 28 小时

Ethereum 主网（出块 ~12 秒）：
  10000 区块 ≈ 33 小时
```

6.2 合约参数（只读）

参数名	典型值	精度	说明	位置
borrowCollateralFactor	0.80	18 decimals	借款抵押率（80% LTV）	assetConfig
liquidateCollateralFactor	0.85	18 decimals	清算触发阈值（85%）	assetConfig
liquidationFactor	0.95	18 decimals	清算激励因子（5% 折扣）	全局配置

storeFrontPriceFactor	0.50	18 decimals	buyCollateral 折扣系数	全局配置

三个抵押率的关系：

```
代码块

1   正常借款 → 接近清算 → 触发清算 → 清算完成
2     80%        83%        85%        清零
3   （借款）    （预警）    （清算）    （债务）
```

健康因子计算：

```
代码块

1   healthFactor = (collateralValue × 0.85) / debtValue
2
3   healthFactor > 1.0: 安全
4   healthFactor < 1.0: 可清算
```

6.3 监听事件类型

事件名	触发场景	提取字段	说明
Withdraw	用户借款/提取 USDC	src, dst	最主要的借款信号
Supply	用户存入 USDC	from, dst	可能有借款人还款
SupplyCollateral	用户存入抵押品	from, dst	借款前的准备动作
WithdrawCollateral	用户提取抵押品	src, to	可能触发清算条件

6.4 日志示例

```
代码块

1   [2025-01-06T10:30:15.000Z] Block: 12345678
2   📊 Querying events from block 12335678 to 12345678...
3     - Withdraw events: 150
4     - Supply events: 80
5     - SupplyCollateral events: 120
6     - WithdrawCollateral events: 90
7   ✅ Found 85 unique addresses from all events
```

```
🔍 Checking 85 addresses for liquidation...
💰 Liquidatable: 0xAlice...1234
💰 Liquidatable: 0xBob...5678
💰 Liquidatable: 0xEve...9012

🎯 Found 3 liquidatable accounts
📤 Sending liquidation transaction...
🔗 Transaction sent: 0xabc123...
✅ Liquidation successful!
   Gas used: 450000
   Block: 12345679
```

---

7. 完整脚本

index.ts

代码块

```
import hre from 'hardhat';
import { liquidateUnderwaterBorrowers } from './liquidateUnderwaterBorrowers';
import * as fs from 'fs';
import * as path from 'path';
const LOOP_DELAY = 5000; // 5 秒轮询间隔

async function main() {
  const network = hre.network.name;
  const chainId = hre.network.config.chainId;

  console.log('\n==========================================');
  console.log('🤖 YT Lending Liquidation Bot Started');
  console.log('==========================================');
  console.log('Network:', network);
  console.log('Chain ID:', chainId);
  console.log('Loop Delay:', LOOP_DELAY, 'ms\n');

  const deploymentsPath = path.join(__dirname, '../../deployments-lending.json');
  if (!fs.existsSync(deploymentsPath)) {
    throw new Error('deployments-lending.json not found');
  }

  const deployments = JSON.parse(fs.readFileSync(deploymentsPath, 'utf-8'));
  const deployment = deployments[chainId?.toString() || '421614'];

```

```
  if (!deployment) {
    throw new Error(`No deployment found for chainId: ${chainId}`);
  }

  console.log('📋 Contract Addresses:');
  console.log('  Lending Proxy:', deployment.lendingProxy);
  console.log('  Base Token (USDC):', deployment.usdcAddress);
  console.log('');

  const [signer] = await hre.ethers.getSigners();
  console.log('👤 Liquidator Address:', await signer.getAddress());
  console.log('💰 Liquidator Balance:', hre.ethers.formatEther(await hre.ethers.provider.getBalance(signer)), 'ETH\n');

  const lendingContract = await hre.ethers.getContractAt(
    'Lending',
    deployment.lendingProxy,
    signer
  );

  console.log('✅ Contracts initialized\n');
  console.log('========================================');
  console.log('🔄 Starting main loop...\n');

  let lastBlockNumber: number | undefined;

  while (true) {
    try {
      const currentBlockNumber = await hre.ethers.provider.getBlockNumber();
      console.log(`[${new Date().toISOString()}] Block: ${currentBlockNumber}`);

      if (currentBlockNumber !== lastBlockNumber) {
        lastBlockNumber = currentBlockNumber;
        await liquidateUnderwaterBorrowers(lendingContract, signer);
        console.log('');
      } else {
        console.log(`Block already checked; waiting ${LOOP_DELAY}ms...\n`);
      }

      await new Promise(resolve => setTimeout(resolve, LOOP_DELAY));
    } catch (error) {
      console.error('❌ Error in main loop:', error);
      console.log(`Retrying in ${LOOP_DELAY}ms...\n`);
      await new Promise(resolve => setTimeout(resolve, LOOP_DELAY));
    }
  }
```

```
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error('❌ Fatal error:', error);
    process.exit(1);
  });
```

liquidateUnderwaterBorrowers.ts

代码块

```
import hre from 'hardhat';
import { Signer } from 'ethers';

const LOOKBACK_BLOCKS = 10000;

export async function getUniqueAddresses(lendingContract: any): Promise<string[]> {
  const currentBlock = await hre.ethers.provider.getBlockNumber();
  const fromBlock = Math.max(currentBlock - LOOKBACK_BLOCKS, 0);

  console.log(`📊 Querying events from block ${fromBlock} to ${currentBlock}...`);

  const uniqueAddresses = new Set<string>();

  try {
    const withdrawEvents = await lendingContract.queryFilter(
      lendingContract.filters.Withdraw(), fromBlock, currentBlock
    );
    for (const event of withdrawEvents) {
      if (event.args?.src) uniqueAddresses.add(event.args.src);
      if (event.args?.dst) uniqueAddresses.add(event.args.dst);
    }
    console.log(`  - Withdraw events: ${withdrawEvents.length}`);
  } catch (error) {
    console.error('  ⚠️  Failed to query Withdraw events:', error);
  }

  try {
    const supplyEvents = await lendingContract.queryFilter(
      lendingContract.filters.Supply(), fromBlock, currentBlock
    );
    for (const event of supplyEvents) {
```

```
      if (event.args?.from) uniqueAddresses.add(event.args.from);
      if (event.args?.dst) uniqueAddresses.add(event.args.dst);
    }
    console.log(`  - Supply events: ${supplyEvents.length}`);
  } catch (error) {
    console.error('  ⚠️  Failed to query Supply events:', error);
  }

  try {
    const supplyCollateralEvents = await lendingContract.queryFilter(
      lendingContract.filters.SupplyCollateral(), fromBlock, currentBlock
    );
    for (const event of supplyCollateralEvents) {
      if (event.args?.from) uniqueAddresses.add(event.args.from);
      if (event.args?.dst) uniqueAddresses.add(event.args.dst);
    }
    console.log(`  - SupplyCollateral events: ${supplyCollateralEvents.length}`);
  } catch (error) {
    console.error('  ⚠️  Failed to query SupplyCollateral events:', error);
  }

  try {
    const withdrawCollateralEvents = await lendingContract.queryFilter(
      lendingContract.filters.WithdrawCollateral(), fromBlock, currentBlock
    );
    for (const event of withdrawCollateralEvents) {
      if (event.args?.src) uniqueAddresses.add(event.args.src);
      if (event.args?.to) uniqueAddresses.add(event.args.to);
    }
    console.log(`  - WithdrawCollateral events: ${withdrawCollateralEvents.length}`);
  } catch (error) {
    console.error('  ⚠️  Failed to query WithdrawCollateral events:', error);
  }

  console.log(`✅ Found ${uniqueAddresses.size} unique addresses from all events`);
  return Array.from(uniqueAddresses);
}

export async function liquidateUnderwaterBorrowers(
  lendingContract: any,
  signer: Signer
): Promise<boolean> {
  const uniqueAddresses = await getUniqueAddresses(lendingContract);

```

```
  if (uniqueAddresses.length === 0) {
    console.log('ℹ️  No active addresses found');
    return false;
  }

  console.log(`🔍 Checking ${uniqueAddresses.length} addresses for liquidation...`);

  const liquidatableAccounts: string[] = [];

  for (const address of uniqueAddresses) {
    try {
      const isLiquidatable = await lendingContract.isLiquidatable(address);
      if (isLiquidatable) {
        console.log(`💰 Liquidatable: ${address}`);
        liquidatableAccounts.push(address);
      }
    } catch (error) {
      console.error(`Error checking ${address}:`, error);
    }
  }

  if (liquidatableAccounts.length > 0) {
    console.log(`\n🎯 Found ${liquidatableAccounts.length} liquidatable accounts`);
    console.log('📤 Sending liquidation transaction...');

    try {
      const liquidatorAddress = await signer.getAddress();
      const tx = await lendingContract.connect(signer).absorbMultiple(
        liquidatorAddress,
        liquidatableAccounts
      );

      console.log(`🔗 Transaction sent: ${tx.hash}`);
      const receipt = await tx.wait();
      console.log(`✅ Liquidation successful!`);
      console.log(`   Gas used: ${receipt.gasUsed.toString()}`);
      console.log(`   Block: ${receipt.blockNumber}`);

      return true;
    } catch (error) {
      console.error('❌ Liquidation transaction failed:', error);
      return false;
    }
  } else {
    console.log('✅ No liquidatable accounts found');
```

```
      return false;
    }
  }
```

---

8. 运行和部署

8.1 开发模式（前台运行）

代码块

```
npx hardhat run scripts/liquidation_bot/index.ts --network arbSepolia
```

8.2 生产部署（PM2 后台运行）

代码块

```
# 1. 安装 PM2
npm install -g pm2

# 2. 启动清算机器人
pm2 start scripts/liquidation_bot/index.ts \
  --name ytlp-liquidation-bot \
  --interpreter npx \
  --interpreter-args "hardhat run --network arbSepolia"

# 3. 查看日志
pm2 logs ytlp-liquidation-bot --lines 100

# 4. 保存配置（开机自启）
pm2 save
pm2 startup

# 5. 监控管理
pm2 list                          # 列出所有进程
pm2 restart ytlp-liquidation-bot  # 重启
pm2 stop ytlp-liquidation-bot     # 停止
pm2 delete ytlp-liquidation-bot   # 删除
```

8.3 参数调优指南

场景 1：测试网（低活跃度）

代码块

```
const LOOP_DELAY = 5000;        // 5 秒（稳定）
const LOOKBACK_BLOCKS = 50000; // ~14 小时（覆盖更多历史）
```

场景 2：主网（高活跃度）

代码块

```
const LOOP_DELAY = 2000;        // 2 秒（快速响应）
const LOOKBACK_BLOCKS = 10000; // ~2.7 小时（用户活跃，足够）
```

总结

本清算机器人采用无状态轮询设计，通过监听多种事件提高覆盖率，对所有 `isLiquidatable` 为 true 的账户无金额限制地执行清算，确保及时清算水下账户，保护协议免受坏账风险。